Linux Server Hardening

A checklist for reducing the attack surface of a Linux server · Debian/Ubuntu and RHEL-family commands shown · AWS notes where they apply

Principles. (1) **Less is safer** - every package, open port, account and privilege is something to attack. (2) **Patch fast** - most compromises use known, already-fixed bugs. (3) **Least privilege** - users, services and cloud roles get only what they need. (4) **Defense in depth** - assume each layer fails and make the next one count. (5) **Restrict outbound traffic too** - a box that cannot reach the internet cannot talk to a C2 server or exfiltrate data easily.

Order of work

#	Area	Biggest win
1	Updates and minimal install	Automatic security updates; remove what you do not use
2	Accounts and SSH	Keys only, no root login, MFA, no SSH open to the world
3	Network and firewall	Default-deny inbound **and** outbound
4	Kernel and filesystem	sysctl hardening, noexec temp dirs, encryption
5	Mandatory access control	SELinux / AppArmor enforcing
6	Services and applications	systemd sandboxing, non-root services, secrets handling
7	Cloud (AWS)	IMDSv2, least-privilege role, SSM instead of SSH
8	Verify and keep verifying	Lynis / CIS benchmark, external scan, re-check regularly

1 - Updates and a minimal install

- ☐ Start from a minimal, supported image (no desktop, no extras). Track the distro's end-of-life date.
- ☐ Enable **automatic security updates**: `unattended-upgrades` (Debian/Ubuntu) or `dnf-automatic` (RHEL/Fedora/Amazon Linux).
- ☐ Reboot after kernel and libc updates - `needrestart` shows what still runs old code; consider kernel live patching (Ubuntu Livepatch, kpatch) where reboots are hard.
- ☐ List and remove packages and services you do not need: `systemctl list-unit-files --state=enabled`, `ss -tulpn`.
- ☐ Remove compilers, `netcat`, `nmap` and similar from production hosts where possible - they are an attacker's toolkit.
- ☐ Use packages only from trusted, signed repositories; do not pipe `curl ... | sh` on servers.

```
# Debian / Ubuntu
sudo apt install unattended-upgrades needrestart && sudo dpkg-reconfigure -plow unattended-upgrades
# RHEL / Fedora / Amazon Linux
sudo dnf install dnf-automatic && sudo systemctl enable --now dnf-automatic-install.timer
```

2 - Accounts, authentication and SSH

- ☐ One named account per person; no shared logins. Lock or delete unused and default accounts (`passwd -l`, `userdel`).
- ☐ Make sure only `root` has UID 0: `awk -F: '$3==0' /etc/passwd`.
- ☐ Grant admin rights with **sudo** to specific users or groups; avoid `NOPASSWD` and `ALL` where narrower rules work.

- ☐ Disable **password** and **root** SSH logins - use keys only. Prefer hardware-backed keys (`ed25519-sk` with a FIDO2 token) or add MFA via PAM.
- ☐ Restrict who may log in (`AllowGroups`) and reduce brute-force noise with **fail2ban** or **CrowdSec**.
- ☐ Protect private keys with passphrases; audit every `~/.ssh/authorized_keys` regularly.
- ☐ Set a password policy for accounts that still have passwords (`pam_pwquality`, `pam_faillock`).

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AuthenticationMethods publickey
AllowGroups ssh-users
MaxAuthTries 3
LoginGraceTime 20
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no          # unless you need tunnels
ClientAliveInterval 300
ClientAliveCountMax 2
# then: sudo sshd -t && sudo systemctl reload ssh   (sshd on RHEL)
```

3 - Network and firewall

- ☐ **Default-deny inbound.** Open only the ports the service needs (`nftables`, `ufw`, `firewalld`) - and the same in AWS security groups.
- ☐ **Never expose SSH to 0.0.0.0/0.** Reach admin access through a VPN (WireGuard), a bastion host, or AWS SSM Session Manager (no inbound port at all).
- ☐ **Default-deny outbound** where you can: allow only the destinations the server needs (package mirrors, APIs, DNS). This is the single best control against C2 beaconing and data exfiltration. On AWS use security-group egress rules, AWS Network Firewall, or an egress proxy.
- ☐ Bind internal services (databases, Redis, admin panels, metrics) to `127.0.0.1` or the private interface - check with `ss -tulpn`.
- ☐ Disable IPv6 only if unused *and* firewall it either way - an unfirewalled IPv6 address is a common blind spot.
- ☐ Put internet-facing web apps behind a reverse proxy / load balancer, optionally with a WAF.

```
# /etc/nftables.conf - minimal default-deny example (adjust ports and egress)
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    iif lo accept
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept
    tcp dport { 22, 443 } ct state new accept      # 22 only from VPN/bastion ideally
  }
  chain forward { type filter hook forward priority 0; policy drop; }
  chain output {
    type filter hook output priority 0; policy drop;
    ct state established,related accept
    oif lo accept
    udp dport 53 accept; tcp dport 53 accept       # DNS
    udp dport 123 accept                           # NTP
    tcp dport { 80, 443 } accept                   # tighten to known destinations
  }
}
```

4 - Kernel and filesystem

- ☐ Apply kernel hardening settings (below) and reload with `sudo sysctl --system`.

- ☐ Mount `/tmp`, `/var/tmp` and `/dev/shm` with `nodev,nosuid,noexec` - attackers love dropping payloads there.
- ☐ Use separate partitions (or at least quotas) for `/var`, `/var/log` and `/home` so a full disk cannot stop logging.
- ☐ Encrypt disks (LUKS; on AWS, default **EBS encryption** for the account).
- ☐ Audit SUID/SGID binaries (`find / -xdev -perm /6000 -type f`) and remove the bit where not needed.
- ☐ Find world-writable files and directories without the sticky bit and fix them.
- ☐ Enable **Secure Boot** where supported; it stops unsigned kernel modules from loading.
- ☐ Disable kernel modules you never need (USB storage, firewire, rare filesystems and protocols) in `/etc/modprobe.d/`.

```
# /etc/sysctl.d/90-hardening.conf
kernel.kptr_restrict = 2                # hide kernel addresses
kernel.dmesg_restrict = 1               # dmesg for root only
kernel.yama.ptrace_scope = 2            # only root can ptrace (stops credential scraping)
kernel.unprivileged_bpf_disabled = 1
net.core.bpf_jit_harden = 2
kernel.kexec_load_disabled = 1
kernel.sysrq = 0
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
fs.protected_fifos = 2
fs.protected_regular = 2
fs.suid_dumpable = 0
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv6.conf.all.accept_redirects = 0
net.ipv4.conf.all.log_martians = 1
```

Test before rolling out: `ptrace_scope = 2` breaks debuggers for normal users, and very strict settings can affect some software.

5 - Mandatory access control

- ☐ **RHEL-family:** keep **SELinux enforcing** (`getenforce`). Fix denials with booleans and labels (`setsebool`, `semanage fcontext`, `audit2why`) - never by switching to permissive.
- ☐ **Debian/Ubuntu/SUSE:** keep **AppArmor** enabled (`aa-status`) and put internet-facing services into enforce mode; write profiles for custom daemons.
- ☐ MAC limits what a compromised service can touch even after the attacker gets code execution.

6 - Services and applications

- ☐ Run every service as its own **unprivileged user**, never as root; bind privileged ports with `CAP_NET_BIND_SERVICE` or a proxy.
- ☐ Sandbox services with systemd and score them with `systemd-analyze security` (lower is better).
- ☐ Keep secrets out of world-readable files, git and environment dumps; use a secret store (AWS Secrets Manager / Parameter Store, Vault) and `0600` permissions where files are unavoidable.
- ☐ Never expose databases, caches, message queues or admin UIs to the internet; require authentication even internally.
- ☐ Web: current TLS only (1.2+), security headers, upload directories not executable, keep frameworks and plugins updated.
- ☐ Containers: rootless where possible, no `--privileged`, no Docker socket mounted into containers, read-only root filesystem, scan images.

```
# systemctl edit myapp.service  - sandboxing drop-in
[Service]
User=myapp
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
RestrictNamespaces=yes
LockPersonality=yes
MemoryDenyWriteExecute=yes     # not for JIT runtimes (Java, Node)
CapabilityBoundingSet=
SystemCallFilter=@system-service
ReadWritePaths=/var/lib/myapp
```

7 - Cloud specifics (AWS)

- ☐ Require **IMDSv2** on every instance (`HttpTokens=required`, hop limit 1) - blocks the classic SSRF-to-credentials attack.
- ☐ Give the instance role **least privilege**; no `*:*`, no long-lived access keys on the box.
- ☐ Use **SSM Session Manager** instead of SSH where possible - no open port, and sessions are logged.
- ☐ Tight security groups (inbound and outbound); private subnets for anything not public; use VPC endpoints instead of internet routes for AWS APIs.
- ☐ Enable **Amazon Inspector** for vulnerability scanning, and **GuardDuty** for threat detection.
- ☐ Default EBS encryption on; block public S3 access at the account level; MFA on all IAM users; no root account use.
- ☐ Automated, **immutable backups** (AWS Backup with Vault Lock, or S3 Object Lock) in a separate account.

```
aws ec2 modify-instance-metadata-options --instance-id i-xxxxxxxx \
    --http-tokens required --http-put-response-hop-limit 1 --http-endpoint enabled
```

8 - Integrity, verification and upkeep

- ☐ Install a file integrity monitor: **AIDE** (`aide --init`, daily `aide --check`) and keep its database off the box.
- ☐ Run **Lynis** (`sudo lynis audit system`) and fix findings; for formal baselines use the **CIS Benchmark** for your distro via **OpenSCAP** or CIS-hardened AMIs.
- ☐ Scan from the **outside** (`nmap -sS -sV -p- <public-ip>`) - only what you intended should answer.
- ☐ Make the server reproducible (Ansible, cloud-init, Packer, Terraform): hardening applied by code is not forgotten on the next box, and a compromised box can simply be rebuilt.
- ☐ Review regularly: users and keys, open ports, sudo rules, installed packages, IAM role permissions.
- ☐ Pair this list with the **forensic-readiness logging** checklist - hardening lowers the odds of a breach, logging decides whether you can understand one.

References: CIS Benchmarks, DISA STIGs, NIST SP 800-123 (Server Security), Lynis, AWS Well-Architected Security Pillar.